

HydraShield — Technical Build Blueprint

Architecture, Workflow, Phase Plan, Datasets & Validation, Impact, USP, and a Start-to-End Build Guide

For the engineering collaborator · companion to the problem-statement & strategy document · August 2026

1. Solution Overview

HydraShield is a graph-native supply-chain intelligence system built on HydraDB. It models the npm/PyPI dependency ecosystem as a version-aware graph and answers one core question in real time: when a specific package version is compromised, exactly which repositories, services, and production environments are exposed, and through what path. Every answer is an explainable graph traversal, not a similarity match or an LLM guess.

This document is the implementation companion to the strategy document already shared. It assumes that document's problem statement and differentiation logic, and focuses entirely on how we build it: architecture, workflow, data model, tech stack, phased task breakdown, datasets, validation methodology, and a literal setup-to-submission build guide.

2. System Architecture

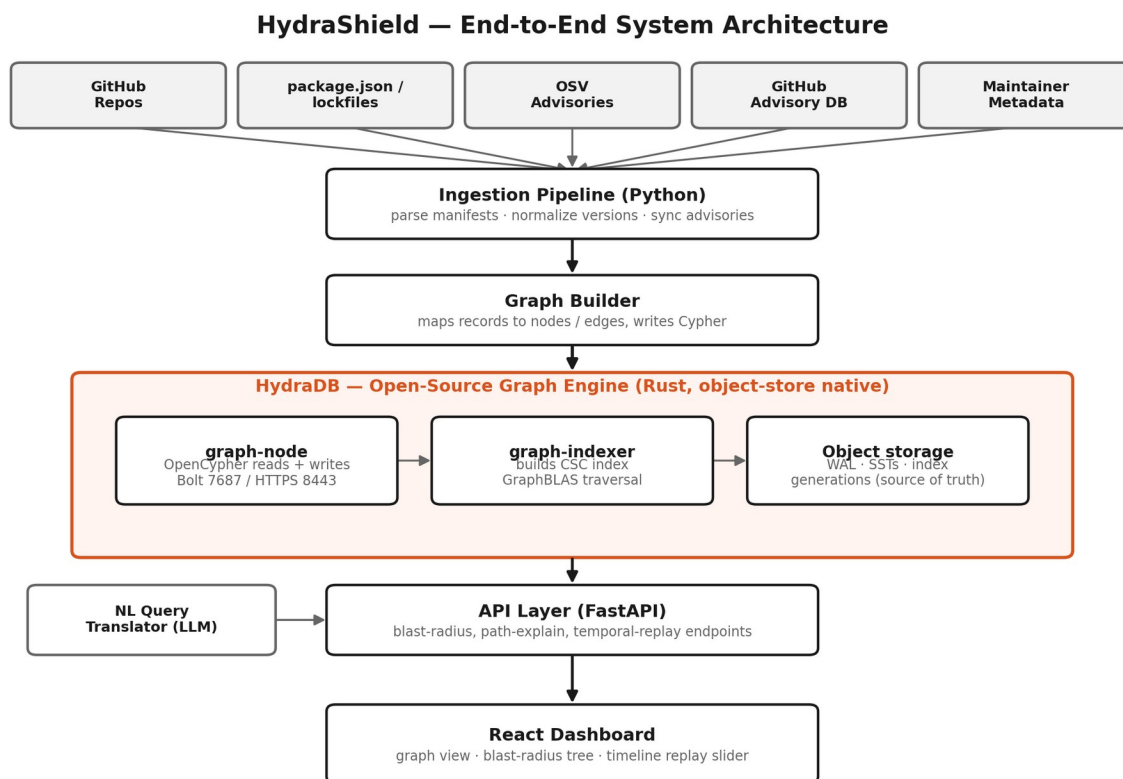


Figure 1 — End-to-end system architecture

Five layers, in order:

- **Data sources:** GitHub repositories, package manifests and lockfiles (package.json, package-lock.json, pnpm-lock.yaml, poetry.lock, requirements.txt), OSV advisories, the GitHub Advisory Database, and package maintainer metadata.
- **Ingestion pipeline (Python):** parses manifests, resolves and normalizes version strings (semver ranges → exact versions), syncs advisory feeds, and hands clean records to the graph builder.
- **Graph builder:** maps normalized records onto the node/edge schema (Section 4) and writes them into HydraDB as Cypher CREATE/MERGE statements — batched via UNWIND for throughput, not one write per node.
- **HydraDB (the graph engine itself):** open-source, written in Rust, object-storage-native. Internally split into **graph-node** (serves OpenCypher reads/writes over Bolt 5.x or HTTPS), **graph-indexer** (builds immutable GraphBLAS-backed traversal indexes in the background), and **S3-compatible object storage** as the single durable source of truth for the WAL, SSTs, and index generations. This is described in full in Section 3.
- **API layer (FastAPI) and React dashboard:** the API exposes blast-radius, path-explanation, and (if built) temporal-replay endpoints, each backed by a Cypher query against graph-node — never by pulling data out and walking it in Python. The dashboard renders the traversal result as a tree and, for the stretch goal, a timeline slider. An optional NL layer translates a plain-English question into query parameters; it never touches the graph directly.

3. HydraDB — Engine Detail (what we're actually deploying)

Two different things carry the HydraDB name. The hackathon hands you the open-source graph engine at github.com/hydra-db/hydradb — a self-hosted Rust database you run yourself. HydraDB's hosted SaaS product (docs.hydradb.com) is a different, higher-level product built on top of a similar engine, oriented around document/memory ingestion for AI agents. We are using the open-source engine directly, via its native Cypher interface — not the hosted ingest/query SDK, which doesn't give you a custom node/edge schema.

3.1 What's actually in the open-source repo

- **Language & storage:** Rust. Durable graph storage on SlateDB, backed by S3-compatible object storage as the single source of truth (WAL, SSTs, manifests, immutable index generations).
- **Disaggregated compute:** graph-node serves queries and canonical mutations; graph-indexer builds traversal indexes (compiled CSC generations) asynchronously in the background using SuiteSparse GraphBLAS. Both roles keep only disposable state locally (SSD/NVMe cache) and can be scaled or replaced independently.
- **Query language:** a practical OpenCypher subset — typed relationships, bounded variable-length paths, property/label predicates, ordering, pagination, aggregation, OPTIONAL MATCH, UNION, and batched UNWIND writes. This is the same query language Neo4j uses.
- **Native path procedures:** algo.SPpaths (single source → single target), algo.SSpaths (single source, all reachable targets within a bound), and algo.MSpaths (many indexed sources and targets evaluated together, avoiding client-side query fan-out). MSpaths is the natural fit for blast radius: one call can resolve “from this compromised version, to every production environment” instead of one query per repository.
- **Connectivity:** Neo4j-compatible Bolt on port 7687 (use any standard Neo4j driver, e.g. the Python neo4j package), or a JSON/NDJSON HTTPS query API on port 8443.

- **Consistency:** causal (default — fast, uses the node's current durable view) or strong (refreshes from object storage before pinning the query snapshot, at extra latency cost). Use causal for the dashboard; consider strong only where a just-written advisory must be visible immediately, e.g. inside the held-out evaluation harness.
- **License:** AGPL-3.0. Fine for a hackathon submission; worth a one-line note in the README since we're distributing our own code alongside it.

Why this matters for our build

Traversal must be expressed as Cypher (ideally the native path procedures) and executed inside graph-node — not fetched into Python and walked with networkx or a hand-written BFS. That's the difference between “using HydraDB meaningfully” and using it as a JSON store, which the hackathon rules explicitly penalize.

The exact parameter shape of algo.MSpaths should be confirmed against docs/architecture.md and the procedure's real signature on Day 1 — treat the Cypher in Section 8 as the intended shape to validate, not a guaranteed-correct call copied blind.

4. Graph Data Model

PackageVersion, not Package, is the central node — vulnerabilities bind to version ranges, and a graph that only knows “lodash” can't distinguish an exposed service from a patched one.

Node	Key properties
Organization	id, name
Repository	id, name, org_id
Service	id, name
Environment	id, name (staging / production)
Package	id, name, ecosystem (npm / pypi)
PackageVersion	id, package_id, version, published_at
Maintainer	id, username, email_hash
Advisory	id, source (OSV / GHSA), summary, published_at
CVE	id, severity, affected_range
Lockfile	id, repo_id, commit_sha, generated_at
Commit	sha, repo_id, author, timestamp
Build	id, repo_id, started_at, status
Deployment	id, service_id, env_id, deployed_at

Edge	Meaning
Repository -USES-> Lockfile	which lockfile is currently active in a repo
Lockfile -RESOLVES_TO-> PackageVersion	exact version a lockfile pinned

Edge	Meaning
PackageVersion -DEPENDS_ON-> PackageVersion	direct dependency edge (the one we traverse in reverse)
PackageVersion -VERSION_OF-> Package	groups versions under a package
Package -MAINTAINED_BY-> Maintainer	ownership, for the maintainer trust graph
PackageVersion -HAS_CVE-> CVE	binds a vulnerability to an exact version
Repository -DEPLOYED_TO-> Environment	where a repo's build actually runs
Commit -INTRODUCED-> PackageVersion	who/when a dependency bump landed
Build -PRODUCED-> Lockfile	provenance of a lockfile
Service -RUNS-> Repository	service-to-code mapping

5. Workflow — Answering a Blast-Radius Question

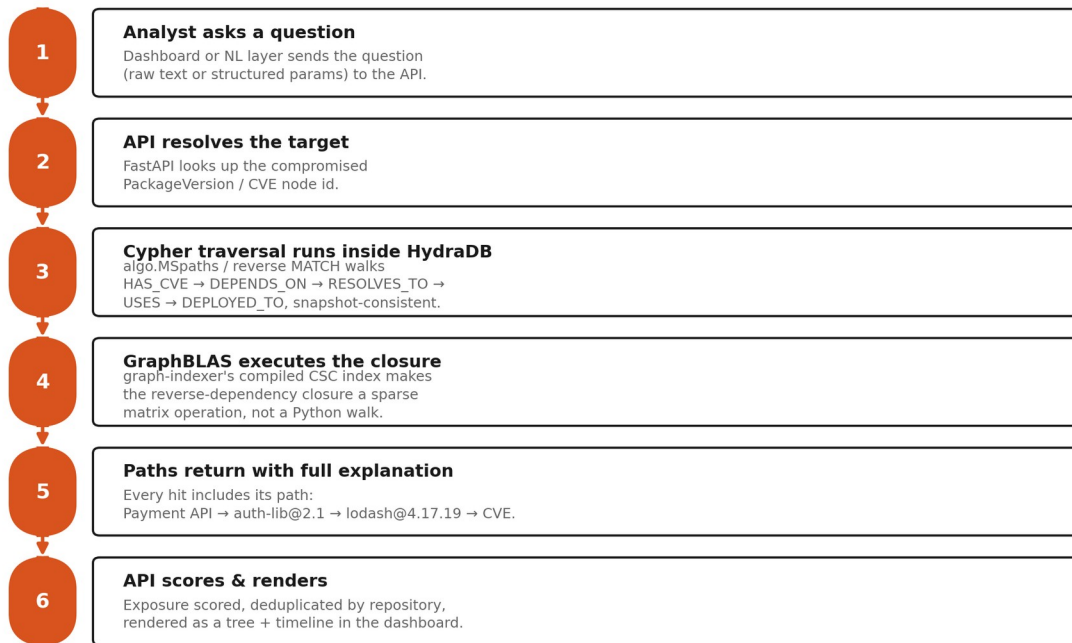


Figure 2 — Request lifecycle for a blast-radius query

The traversal itself (steps 3–4) happens entirely inside HydraDB. The API layer's job is to resolve the starting node, pass parameters to a Cypher query or path procedure, and format what comes back — it should contain no graph-walking logic of its own. This is the single most important engineering discipline for this project; see the callout in Section 3.

6. Full Technology Stack

Layer	Technology	Notes
Graph database	HydraDB (open source, Rust)	Self-hosted via Docker or built from source; Bolt + HTTPS
Graph driver	neo4j Python driver / Bolt client	HydraDB is Bolt-5.x compatible — no custom driver needed
Ingestion	Python 3.11+	Manifest/lockfile parsing, OSV + GHSA sync, version normalization
Manifest parsing	npm: @npmcli/arborist or a lockfile parser lib; Python: packaging + tomllib for poetry.lock	Resolve exact versions, not ranges
Vulnerability data	OSV API / bulk export, GitHub Advisory Database (git-clonable OSV-format JSON)	Ground truth for CVEs and affected ranges
Backend API	FastAPI + Pydantic	Async endpoints; thin layer over Cypher, no traversal logic
Background jobs	asyncio / Celery (optional)	Advisory polling, snapshotting for temporal replay
Frontend	React (Vite) + React Flow / Cytoscape.js	Graph view, blast-radius tree, timeline slider
Charts / timeline	D3.js	Temporal replay visualization if built
NL query layer (optional)	Claude or GPT via API	Translates question → query params only; never reasons over the graph itself
Testing	pytest, networkx (for independent baseline checks)	Unit tests for parsers, regression checks for traversal correctness
Containerization	Docker / docker-compose	Local dev matches the README's single-node setup
Deployment (optional)	Kubernetes via the HydraDB Helm chart	Only if going beyond a single dev node

7. Phase-Wise Work Distribution

Roles are split by what each of you is best positioned to own — adjust freely. “Product/Data” tasks don't require deep systems programming; “Engineering” tasks are where the tech person's time should concentrate.

Phase 0 — Setup & Scoping — Day 1

Goal: Stand up HydraDB locally, lock the schema, make the A-vs-B track call final.

Product / Data (you):

- Confirm Direction A (blast radius) vs B (code graphs) as a deliberate decision, not a default.
- Draft the README skeleton and the demo video outline (problem → project → demo → HydraDB usage, per the submission requirements).
- Start sourcing the dataset shortlist (Section 9).

Engineering (tech collaborator):

- Stand up a local HydraDB dev node via Docker (see Section 10.1) and confirm a round-tripped write/read.
- Finalize the node/edge schema from Section 4 as actual Cypher CREATE CONSTRAINT / property definitions.
- Set up the repo skeleton (Section 11).

Exit criteria: *A running HydraDB node answers a hand-written CREATE + MATCH query; schema and repo structure are committed.*

Phase 1 — Ingestion — Day 2

Goal: Get real dependency and advisory data flowing into HydraDB.

Product / Data (you):

- Curate the initial package/repository sample (Section 9.1) and confirm licensing/attribution needs for the README.
- Validate a handful of ingested records manually against the source data.

Engineering (tech collaborator):

- Build manifest/lockfile parsers for npm and PyPI.
- Build the OSV + GitHub Advisory Database sync job.
- Write the graph-builder layer that turns parsed records into batched Cypher writes.

Exit criteria: *A real (not synthetic) slice of the npm/PyPI ecosystem is loaded into HydraDB with CVEs attached.*

Phase 2 — Graph Construction at Scale — Day 3

Goal: Scale ingestion to the full snapshot the evaluation will run against.

Product / Data (you):

- Decide the snapshot cutoff date (Section 9.2) and freeze it.
- Spot-check graph completeness against known packages.

Engineering (tech collaborator):

- Run full ingestion against the frozen snapshot.
- Add indexes/constraints for query performance (property indexes on PackageVersion.name+version, CVE.id).
- Confirm graph-indexer is keeping up (check CSC generation lag).

Exit criteria: *The full historical snapshot is loaded and query latency is sane on a cold cache.*

Phase 3 — Blast Radius Traversal — Day 4

Goal: Native Cypher traversal answering the brief's exact example query.

Product / Data (you):

- Write out 10–15 concrete test questions (Section 5's example plus edge cases) for the engineering team to validate against.
-

Engineering (tech collaborator):

- Implement the reverse-dependency Cypher query (Section 8.2) and/or algo.MSpaths call (Section 8.3).
- Implement path-explanation formatting (full chain, not just endpoints).
- Expose both via the FastAPI layer.

Exit criteria: *Given a CVE id, the API returns exposed repositories/services/environments with full explanation paths, matching manual spot-checks.*

Phase 4 — Held-Out Evaluation Harness — Day 5

Goal: Build the thing most competing teams will skip: a real blind test.

Product / Data (you):

- Assemble the held-out advisory set (Section 9.3) and the synthetic ground-truth fixtures (Section 9.4).
- Define what “correct” means for each test case in writing.

Engineering (tech collaborator):

- Build a script that ingests the frozen snapshot, then applies held-out advisories, then runs blast-radius queries and scores precision/recall/latency against known-correct answers.
- Log cost proxies (bytes scanned, tier hit ratio) per query.

Exit criteria: *A single command runs the eval and prints precision, recall, p50/p95 latency, and a cost estimate — numbers go straight into the README.*

Phase 5 — Dashboard & API Polish — Day 6

Goal: A demoable, judge-readable product.

Product / Data (you):

- Define the exact demo script/scenario the video will walk through.
- UX review of the dashboard.

Engineering (tech collaborator):

- Build the React dashboard: search/select a CVE or package, render the blast-radius tree with explanation paths.
- Harden the API (error handling, input validation, OpenAPI docs).

Exit criteria: *The demo scenario runs end-to-end in the UI without a developer intervening.*

Phase 6 — Stretch Features — Days 7–8

Goal: Only attempted if Phase 4's evaluation already passes cleanly.

Product / Data (you):

- Prioritize: temporal replay first (uses HydraDB's native versioned edges directly, see Section 3), maintainer trust graph second.
- Re-run the eval harness after each addition to confirm nothing regressed.

Engineering (tech collaborator):

- Temporal replay: query graph state at time T using valid-time edge properties; build the timeline slider endpoint.
- Maintainer trust graph: MAINTAINED_BY traversal to surface co-maintained packages and their consuming repos.

Exit criteria: *Each stretch feature ships fully working or is cut — no half-built features in the final submission.*

Phase 7 — Packaging & Submission — Day 9

Goal: Everything on the guide's checklist, verified line by line.

Product / Data (you):

- Write the final README (problem, architecture, how HydraDB is used and what's lost without it, setup instructions, eval numbers).
- Record and edit the 3-minute demo video.
- Submit the Google Form.

Engineering (tech collaborator):

- Freeze features. Fix only bugs.
- Confirm setup instructions work on a clean machine/container.
- Tag the release commit.

Exit criteria: *Every item in Section 12 (submission checklist) is checked before 11:59 PM PT, August 20.*

8. Reference Cypher & API Calls

These mirror the real HydraDB HTTP/Cypher surface (verified against the open-source README) — treat them as a starting point to adapt once the schema is finalized in Phase 0.

8.1 Schema writes (batched via UNWIND)

```
UNWIND $versions AS v
MERGE (p:Package {name: v.package, ecosystem: v.ecosystem})
MERGE (pv:PackageVersion {id: v.id})
  ON CREATE SET pv.version = v.version, pv.published_at = v.published_at
MERGE (pv)-[:VERSION_OF]->(p);

UNWIND $deps AS d
MATCH (parent:PackageVersion {id: d.parent_id})
MATCH (child:PackageVersion {id: d.child_id})
MERGE (parent)-[:DEPENDS_ON]->(child);
```

8.2 Reverse-dependency blast radius (plain Cypher)

Illustrative — bind the path in the MATCH to return it; adapt bounded-length syntax to the engine's supported OpenCypher subset.

```
MATCH (cve:CVE {id: $cveId})<-[:HAS_CVE]-(pv:PackageVersion)
```



```

MATCH (repo:Repository)-[:USES]->(:Lockfile)-[:RESOLVES_TO]->(entry:PackageVersion)
MATCH (entry)-[:DEPENDS_ON*0..8]->(pv)
MATCH (repo)-[:DEPLOYED_TO]->(env:Environment)
RETURN DISTINCT repo.name AS exposed_repo, env.name AS environment,
  [n IN nodes(path) | n.version] AS dependency_chain

```

8.3 Using the native path procedure for many-target closure

Verify the exact `MSpaths` parameter contract against `architecture.md` / the procedure's real signature on Day 1 before relying on this shape.

```

CALL algo.MSpaths({
  sourceLabel: 'PackageVersion',
  sourceProperty: 'id',
  sourceValues: [$compromisedVersionId],
  targetLabel: 'Environment',
  targetProperty: 'name',
  targetValues: ['production'],
  relTypes: ['DEPENDS_ON', 'RESOLVES_TO', 'USES', 'DEPLOYED_TO'],
  relDirection: 'both',
  maxLen: 8,
  pathCount: 500,
  resultLimit: 5000
})
YIELD path
RETURN path

```

8.4 HTTP query (no driver needed)

```

curl -sS http://127.0.0.1:8443/v1/graphs/default/query \
-H "Authorization: Bearer $TOKEN" \
-H 'X-Graph-Namespace: default' \
-H 'Content-Type: application/json' \
--data '{"cell_id":"cell-0","query":"MATCH (p:PackageVersion {id:\\"lodash@4.17.19\\"})<-[:DEPENDS_ON*0..6]-(x) RETURN x LIMIT 50"}'

```

9. Dataset Strategy — Build, Test, and Validate

The brief's own evaluation is a blind test: ground truth from OSV/GHSA, recent advisories withheld, teams scored against an older snapshot. Our dataset plan mirrors that structure exactly rather than inventing a separate one.

9.1 Ecosystem data (the graph itself)

- **npm:** start from a curated sample — e.g. the top few thousand packages by download count — rather than the full registry. Resolve each package's transitive dependency tree from its lockfile, not just its declared dependencies, so `DEPENDS_ON` reflects what actually gets installed.

- **PyPI:** PyPI's JSON API (pypi.org/pypi/<name>/json) per package, or the public PyPI BigQuery dataset for bulk metadata if broader coverage is needed.
- **Vulnerabilities:** OSV.dev (API and bulk export, ecosystem-partitioned) plus the GitHub Advisory Database, which is directly git-clonable as OSV-format JSON — both are independent of anything we ingest, which is what makes precision/recall against them meaningful.
- **Maintainers:** the maintainers field from npm's registry API and PyPI's package JSON.

9.2 Snapshot cutoff (the “older ecosystem graph” the brief describes)

Freeze the graph as of a fixed date — a few weeks before the hackathon started is a reasonable default. Everything ingested for the MVP graph comes from before this date. This is what Phase 2 locks in.

9.3 Held-out validation set (the blind test)

Advisories published after the snapshot cutoff, up to the current date, are withheld from initial ingestion and applied later as a simulated “new compromise” during Phase 4. This validates two things at once: that the ingestion pipeline can incrementally add a new CVE/PackageVersion/edge set without a full rebuild, and that the traversal correctly finds every exposed node once it does. This is the part a purely AI-scaffolded submission is likely to skip — see the strategy document, Section 6.2.

9.4 Synthetic ground-truth fixtures (for exact precision/recall)

Public data has no ground truth for “which of our services are exposed” — that depends on a company's own repos, which don't exist publicly. Solve this by generating a small synthetic company graph: 20–50 Repository/Service/Environment nodes, each wired to a real lockfile sampled from public GitHub repos, with a handful of known-vulnerable versions deliberately planted at varying depths (1-hop, 3-hop, 6+ hop). Because we control the fixture, the correct answer is known exactly, which makes precision/recall/traversal-depth testing deterministic instead of estimated.

9.5 Independent regression baseline

On the same small synthetic fixture, compute blast radius independently in Python with networkx and diff it against HydraDB's Cypher result. Divergence here isolates a query-logic bug from a data problem — do this once traversal is implemented (end of Phase 3) and again after any schema change.

10. Validation & Testing Methodology

Test type	What it checks	When
Unit tests (pytest)	Manifest/lockfile parsers produce correct nodes/edges for known fixture files	Phase 1 onward, in CI
Schema regression	Cypher writes create the expected node/edge counts on a fixture graph	After any schema change
Traversal regression	HydraDB result vs. independent networkx result on the synthetic fixture (Section 9.5)	End of Phase 3, before Phase 6
Held-out precision/recall	Blast-radius output vs. OSV/GHSA-declared affected	Phase 4, re-run after every

Test type	What it checks	When
	ranges, on withheld advisories (Section 9.3)	ingestion change
Latency	p50 / p95 / p99 query time at increasing traversal depth and graph size	Phase 4 and again after any indexing change
Cost proxy	Object-storage bytes scanned / hot-warm-cold tier hit ratio per query	Phase 4
End-to-end smoke test	Docker-composed HydraDB + API + ingested fixture answers one known query correctly, single command	Before every demo run and before submission

Report the held-out precision/recall, latency, and cost numbers directly in the README and the demo video — the brief states these are the actual scoring dimensions, not the polish of the walkthrough.

11. Project Repository Structure

```
hydrashield/
├── ingestion/
│   ├── npm_parser.py      # lockfile -> normalized records
│   ├── pypi_parser.py
│   ├── osv_sync.py       # OSV bulk/API sync
│   └── github_advisory_sync.py
├── graph/
│   ├── schema.cypher     # CREATE CONSTRAINT / index definitions
│   ├── graph_builder.py  # records -> batched Cypher writes
│   ├── blast_radius.py   # Section 8.2 / 8.3 queries, called via driver
│   ├── temporal.py       # stretch: valid-time queries
│   └── risk.py           # stretch: maintainer trust graph
├── api/
│   ├── main.py           # FastAPI app
│   ├── routes.py         # /blast-radius, /explain, /replay
│   └── nl_query.py        # optional NL translation layer
├── ui/                   # React app (Vite)
│   ├── src/graph/        # React Flow / Cytoscape views
│   └── src/timeline/      # stretch: replay slider
├── eval/
│   ├── run_eval.py       # Section 10 harness — the differentiator
│   ├── fixtures/         # synthetic ground-truth graph (Section 9.4)
│   └── held_out_advisories/ # Section 9.3 withheld set
├── docker-compose.yml    # HydraDB graph-node + graph-indexer + API
├── tests/                # pytest unit + regression tests
└── README.md             # problem, architecture, HydraDB usage, eval numbers
```

12. Start-to-End Build Guide

12.1 Day-1 local HydraDB setup

Fastest path — the published Docker image (no local Rust toolchain needed):

```
mkdir -p hydradb-data/store hydradb-data/cache
```

```
printf '%s\n' 'local-development-token-32-bytes' > hydradb-data/auth-token
```

```
docker run --rm \
  --user "$(id -u):$(id -g)" \
  -p 7687:7687 -p 8443:8443 -p 9090:9090 \
  -v "$PWD/hydradb-data:/data" \
  -e CLOUD_PROVIDER=local -e LOCAL_PATH=/data/store \
  -e GRAPH_NAMESPACE=default -e GRAPH_ID=default \
  -e GRAPH_CELL_ID=cell-0 -e GRAPH_CELLS=cell-0 \
  -e GRAPH_NODE_ID=node-0 \
  -e GRAPH_BOLT_NODE_ADDRESSES=node-0=127.0.0.1:7687 \
  -e GRAPH_ADVERTISED_BOLT_ADDR=127.0.0.1:7687 \
  -e GRAPH_DATA_CACHE_DIR=/data/cache \
  -e GRAPH_AUTH_TOKEN_FILE=/data/auth-token \
  -e GRAPH_ALLOW_PLAINTEXT=true \
  -e RUST_MIN_STACK=33554432 \
  ghcr.io/hydra-db/hydradb:latest
```

Verify with a round-tripped write, not just a listening port:

```
TOKEN='local-development-token-32-bytes'
curl -sS http://127.0.0.1:8443/v1/graphs/default/query \
  -H "Authorization: Bearer $TOKEN" -H 'X-Graph-Namespace: default' \
  -H 'Content-Type: application/json' \
  --data '{"cell_id":"cell-0","query":"CREATE (a {id: 1})-[:FOLLOWS]->(b {id: 2})"}'

curl -sS http://127.0.0.1:8443/v1/graphs/default/query \
  -H "Authorization: Bearer $TOKEN" -H 'X-Graph-Namespace: default' \
  -H 'Content-Type: application/json' \
  --data '{"cell_id":"cell-0","query":"MATCH (a {id: 1})-[:FOLLOWS]->(b) RETURN b.id AS id"}'
```

The second call should return one row with the node id 2. Once this works, port the same environment variables into `docker-compose.yml` so ``docker compose up`` brings up HydraDB alongside the API for the rest of the team.

12.2 Build order (maps to Section 7 phases)

- 1. Schema first: write `schema.cypher`, create it against the running node, confirm constraints via a manual `MATCH`.
- 2. Ingestion second: get one real repository's lockfile fully into the graph before scaling up — verify node/edge counts by hand.
- 3. Traversal third: implement Section 8.2's query against that one repository, confirm the answer by inspection before trusting it at scale.
- 4. Evaluation fourth: don't build the dashboard before the eval harness exists — the eval harness is what tells you whether the traversal is actually correct once the graph is too big to eyeball.
- 5. UI and stretch features last, and only after the eval harness passes.

12.3 Definition of done (per the hackathon's own guide)

- Public GitHub repo, open-source license, no participant commits dated before August 12, 2026.

- README explains the problem, what was built, and specifically how HydraDB is used and what the project would lose without it.
- Setup instructions work end to end on a machine that has never seen the repo before.
- Demo video ≤ 3 minutes: problem $\rightarrow$ project $\rightarrow$ demo $\rightarrow$ HydraDB usage.
- Submitted via the official Google Form before 11:59 PM PT, August 20, 2026.

13. Impact

The problem HydraShield addresses is measured in minutes, not abstractions: the brief's own reference incident saw 84 malicious artifacts land across 42 packages within six minutes of a CI breach, and spread to over 160 downstream packages. The defender's task in that scenario — knowing which of your own services are exposed before the next deploy — is currently done by manual dependency-tree auditing or blunt, alert-heavy scanners with no path reasoning.

- **Speed:** blast-radius lookup moves from a manual, hours-long audit (or a scanner's flat, unranked alert list) to a graph query answered in the time it takes to render a page.
- **Actionability:** every result carries its causal path, so a responder knows exactly which dependency chain to patch or pin, not just that “something downstream is affected.”
- **Proactive risk, not just reactive alerts:** the maintainer trust graph (if built) surfaces the next likely compromise before it happens, from shared maintainers and infrastructure.
- **Measurable, not aspirational:** because the solution is validated against a held-out advisory set with reported precision, recall, latency, and cost (Section 10), the impact claim is falsifiable — the same rigor a real security team would demand before trusting the tool.
- **Deployable beyond the hackathon:** HydraDB's object-storage-native design means the ecosystem graph doesn't need to fit in memory to stay useful, so the same architecture holds up at real company scale, not just on a demo dataset.

14. USP — What to Say When Pitching This

- **Native graph reasoning, not RAG with extra steps.** Every answer is a traversal path computed inside the database, not a similarity score computed outside it — the kind of multi-hop, structural question a vector index cannot answer at all.
- **Time-aware by construction.** Built on HydraDB's versioned, append-only edges, so the system can answer “when did this exposure begin,” not just “what is exposed right now.”
- **Explainable by default.** Every result ships with its full causal chain (Payment API $\rightarrow$ auth-lib@2.1 $\rightarrow$ lodash@4.17.19 $\rightarrow$ CVE), directly usable by a security engineer without further digging.
- **Verified against a blind test, not just demoed.** Held-out advisories, synthetic ground truth, and reported precision/recall/latency/cost — the same evaluation shape the hackathon itself uses, and the same one a real buyer would ask for.
- **Open and self-hostable.** Built on an AGPL-3.0, Rust, object-storage-native graph engine — no vendor lock-in, deployable inside a company's own cloud account.

15. Final Checklist Before Submission

- HydraDB usage is explicit in the README, with a direct answer to “what would we lose without it.”
- Blast-radius traversal runs as Cypher / native path procedures inside HydraDB, not as Python graph-walking on data pulled out of it.
- Held-out evaluation numbers (precision, recall, latency, cost) are in the README and referenced in the demo video.
- Repo is public, licensed, README explains setup end to end, no commits before August 12, 2026.
- Demo video is ≤ 3 minutes, structured problem $\rightarrow$ project $\rightarrow$ demo $\rightarrow$ HydraDB usage.
- Submission form completed before 11:59 PM PT, August 20, 2026.